Technical report — Redacta

Redacta proposes which words to remove from a Swiss court decision before it is published, and a court clerk approves each one. It runs on the court's own hardware, with no internet connection.

- **Track:** Track 2B — Own Project
- **Event:** Online
- **Team:** Redacta — Raemin Kang
- **Demo:** (link in the submission form)
- **Code:** https://github.com/kangraemin/redacta-apertus (`track_2b/`)

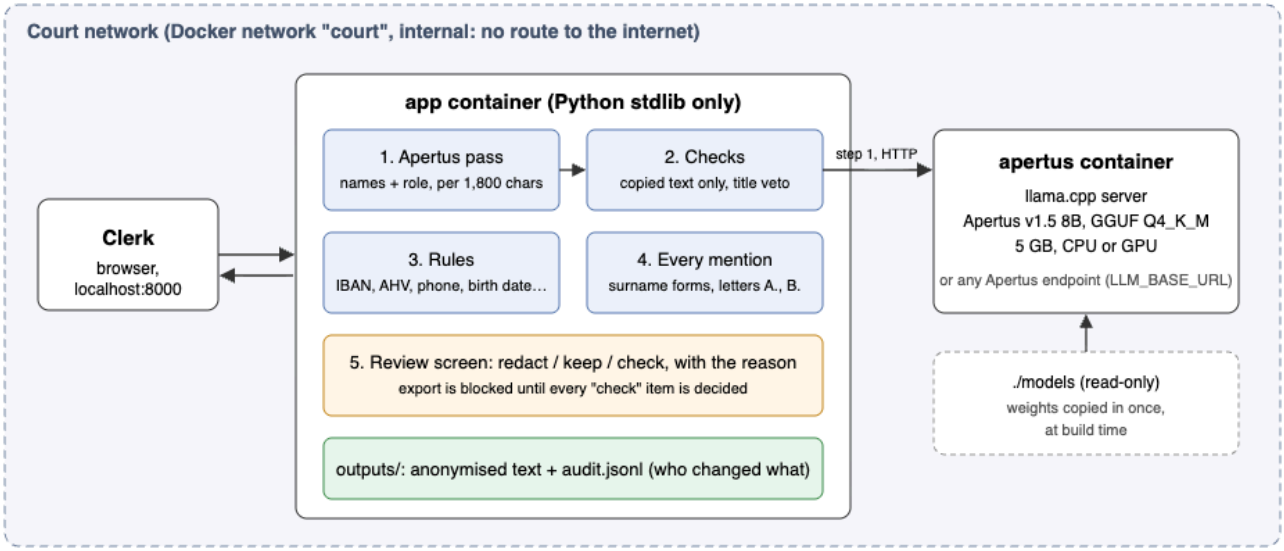
1. Summary

Swiss courts must make their decisions public. Before a decision goes out, someone replaces the names of parties, relatives, witnesses and private companies with letters (`A. ____`), and removes addresses and dates of birth. Judges, clerks, lawyers and authorities stay named. Unpublished decisions are confidential, so the text cannot be sent to a commercial cloud model. Trained staff do this work. At the Federal Supreme Court, paralegals are assisted by a tool (Anom2) that finds entities with regular expressions and other methods, and the court's own study added a BERT model to it (Niklaus et al., 2023). In 1C_394/2018 the Federal Supreme Court held that the Geneva criminal court could not limit access to its decisions on the grounds that anonymising about 22,000 of them was too much work.

Redacta puts Apertus v1.5 8B and a set of fixed rules side by side. Apertus lists every name in each part of the decision and gives it a role (party, private person, judge, lawyer, authority, ...). The rules catch identifiers that have a fixed shape (IBAN, AHV number, phone, licence plate, date of birth, street address) and stop names that follow a judge, clerk or lawyer title from being removed. The clerk sees the decision with every proposal marked, the reason for each one, and a short list of items the system is unsure about. Nothing is exported until those are decided, and every approval is logged.

To measure accuracy without hand labelling, we took 180 published Federal Supreme Court decisions (German, French, Italian), which are already anonymised, and put synthetic names, addresses and birth dates back where the placeholders were. Each inserted string has known offsets, so the answer key is exact. On the 144 test decisions, Redacta by default removes **96.1%** of the 1,044 strings that must go (rules alone: 14.0%; Apertus alone: 82.8%; on the 36 dev decisions: 93.9%). It wrongly removes **1.4%** of the 570 occurrences of judges', clerks' and lawyers' names. An off-the-shelf multilingual NER model, which we ran only as a baseline, finds 72.6% of the strings but also removes 59.0% of those judges', clerks' and lawyers' names, because it sees entity types, not roles. Redacta takes 25.8 s per decision on a laptop GPU (110.4 s on CPU in Docker) and needs no external service. With the community weights that `make model` downloads, the numbers are almost the same (96.2% found, 1.4% wrongly removed).

2. Architecture



`make run` starts two containers with Docker Compose:

Container	What it does	Network
`apertus`	llama.cpp server (`ghcr.io/ggml-org/llama.cpp:server-b11459`) with Apertus v1.5 8B, GGUF Q4_K_M, OpenAI-compatible API	`court` only (internal: no route out)
`app`	Python standard library only: anonymisation pipeline + review web page on `localhost:8000`	`court` + a front door bound to 127.0.0.1

Pipeline for one decision (`src/anonymizer/pipeline.py`):

1. **Apertus pass.** The text is cut into paragraph-aligned chunks of about 1,800 characters. Each chunk goes to Apertus with the decision header (the list of parties) as context. Apertus answers with a JSON list of names and roles. Temperature 0, reasoning switched off (`enable_thinking: false`).
2. **Checks on the model's answer.** A name is used only if it appears verbatim in the chunk, so made-up names are dropped. Roles are voted over all mentions of a name. Common nouns, acronyms and role words are removed.
3. **Rules.** Regular expressions for identifiers with a fixed shape. A *veto* keeps any name that sits in the bench list (`Besetzung / Composition / Composizione`) or directly after a judge, clerk or lawyer title. A list of public bodies (courts, cantons, offices, insurers) is never redacted. An address that is not next to a redacted party goes to "you decide" instead of being removed.
4. **Every mention.** A full name also covers the surname on its own later in the text, and a relative who shares the surname is caught with their first name. Letters are assigned in order of first appearance, as the court does (A, B, C ...).
5. **Review screen.** Three lists: *for you to decide*, *redact*, *keep*, each item with its reason (`Apertus: party (3 mentions)`, `rule: date of birth`, `rule: appears with a judge/clerk/lawyer title`). Clicking a marked word selects its item; `R` and `K` change it, `Z` undoes, the arrow keys move through the list. Role words stay readable (`Gegenpartei B.`_____) and only the date of a birth date is replaced (`geboren am […]`). *Published text* shows the final text. *Approve and export* writes the text and a line in `outputs/audit.jsonl` (time, reviewer, model, what the reviewer changed).

Target architecture (mandatory)

b) Air-gapped, and therefore also **a) on-premise**.

- *Runtime:* no external network. The Apertus container is on a Docker network marked `internal: true`. The app image needs no `pip install`; it uses only the Python standard library. The weights are mounted read-only from `./models`.

- *Build time (once, with internet):* pull two images (`python:3.12-slim`, llama.cpp server) and fetch the 5 GB GGUF (`make model`, or build it from the official weights with `scripts/convert_gguf.sh`). These files can be carried into a closed network on a disk.
- If the court already runs Apertus somewhere (for example on a Swiss sovereign cloud or a CSCS endpoint), setting `LLM_BASE_URL`, `LLM_NAME`, `LLM_API_KEY` makes `make run` start only the app and use that endpoint.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (text branch), GGUF Q4_K_M, 5.1 GB.
- **How it is used:** inference: name finding and role labelling, structured JSON output.
- **Where it runs:** local weights in llama.cpp (CPU or GPU). Our measurements ran on an Apple M1 Max (32 GB) with Metal.

The v1.5 8B checkpoint is multimodal. We kept only the text branch: a text-only config, the first 131,072 rows of the embedding table, and `<|assistant_end|>` (id 68) marked as end-of-generation; without that the model does not stop. `scripts/convert_gguf.sh` reproduces this from the official safetensors. The prompt (`src/anonymizer/llm.py`) states Swiss practice in two sentences (who is replaced, who stays) and asks only for a list of names with roles. The model never rewrites the text. That keeps every change visible and lets the code check each claim against the source.

Why Apertus and not a smaller NER model: the decisive information is the *role*, not the entity type. "Me Christophe Pittet" (lawyer, stays) and "Christophe Pittet, recourant" (party, goes) are the same entity type. Apertus reads the procedural context in all three languages with one model. Fixed-shape identifiers and the title veto are left to rules, because rules are exact there. No other model is used in the system. For the evaluation only, we ran one off-the-shelf NER model (spaCy `xx_ent_wiki_sm`) as a baseline to test exactly this point (Section 5). It is not part of Redacta and is not in the Docker image.

4. Data

- **Source:** 180 decisions of the Swiss Federal Supreme Court (October 2025 – September 2026) from entscheidsuche.ch, as published, i.e. already anonymised by the court. Official decisions are not protected by copyright in Switzerland (Art. 5 URG). `src/fetch_decisions.py` downloads them; the texts are in `data/raw/` (1.4 MB).
- **Refilled benchmark** (`src/build_benchmark.py`, seed 7): every placeholder `A._____` is replaced by a synthetic name from `src/names.py` (common first names and surnames per language region). Placeholders that share a last letter (`B.A._____`, `C.A._____`) get the same surname, as relatives. Later mentions alternate between full name and surname alone. Some parties in the header get a synthetic address or date of birth. Company placeholders (`B._____ AG`) get a company name. Every inserted string is stored with its offsets: 1,306 strings in total (1,019 person, 114 address, 97 org, 63 date of birth, 13 location).
- **Must-not-redact side:** judges, clerks and lawyers are never anonymised in the originals, so their real names give 570 occurrences in the test split that must stay.
- **Split:** every fifth file is *dev* (36 decisions, used while writing prompts and rules), the rest is *test* (144 decisions: 45 DE, 51 FR, 48 IT). All numbers below are on test.
- No personal data: the only real names left are those of judges, clerks and lawyers, which the court itself publishes. Licence of our data files: CDLA-Permissive-2.0.

5. Evaluation

Task: given the refilled decision, which characters must be removed. A gold string counts as found if at least 90% of its characters are inside spans the system marks *redact* by default, i.e. before the clerk touches anything. Items in *you decide* are not counted as found. *Precision* counts the system's redacted spans that overlap some gold string. *Protected redacted* is the share of judge, clerk and lawyer name occurrences that were wrongly marked. *Leak-free decisions* is the share of decisions where every gold string was found.

Setup (test, 144 decisions)	Recall	Precision	Leak-free decisions	Protected redacted	Items to confirm / decision
Rules only (regex)	14.0%	57.9%	0.0%	0.0%	1.8
Apertus only (no rules, no veto)	82.8%	70.0%	19.4%	3.2%	3.3
Off-the-shelf NER, baseline only (spaCy `xx_ent_wiki_sm`, every PER/ORG)	72.6%	19.2%	9.7%	59.0%	15.5
Same NER + Redacta's rules and veto (Apertus replaced by NER)	84.5%	28.6%	42.4%	11.2%	11.7
Redacta (Apertus + rules + veto)	**96.1%**	**95.5%**	**77.8%**	**1.4%**	3.9
Redacta with the weights `make model` downloads (community GGUF, see §8)	96.2%	94.1%	78.5%	1.4%	3.9

The NER rows answer the question whether a small entity tagger would do. With the same rules and veto around it, the tagger finds 84.5% instead of 96.1% and still removes 11.2% of the protected names, mostly in the signature lines ("Le Président : Hurni"), which our title rule does not cover. There Apertus reads the role from the context; the tagger only knows that a person is named. The baseline is `scripts/baseline_ner.py`, run outside Docker with its own requirements (`scripts/requirements-eval.txt`).

Related work. The Federal Supreme Court's own study (Niklaus et al., 2023) fine-tuned BERT-type models on 119,156 rulings annotated with the entities the court anonymised. Recall rose from 83.8% (multilingual BERT) to 92.6% (a model pre-trained on Swiss legal text), with 92.3% precision. Their test data and their metric (entity-level scores on the court's own annotations) differ from ours, so the numbers are not directly comparable. The difference in setup is the point: they needed a large labelled corpus and a model trained for the task; Redacta uses no labelled training data and an open general model, prompted with two sentences of Swiss practice.

By language and type (Redacta):

	Decisions	Recall	Precision	Leak-free	Protected redacted	Person	Organisation	Address	Date of birth
DE	45	94.6%	91.2%	71.1%	0.9%	97.4% (n=190)	80.0% (n=5)	100.0% (n=19)	100.0% (n=19)
FR	51	97.1%	94.9%	80.4%	2.0%	97.2% (n=316)	100.0% (n=36)	100.0% (n=37)	100.0% (n=17)
IT	48	96.0%	99.0%	81.2%	0.0%	95.3% (n=300)	95.1% (n=41)	100.0% (n=40)	100.0% (n=14)
All	144	96.1%	95.5%	77.8%	1.4%	96.5% (n=806)	96.3% (n=82)	100.0% (n=96)	100.0% (n=50)

What goes wrong

We read every miss and every false positive on test (`docs/results/eval_full/errors.jsonl`).

A bug the test set exposed. The first full run gave 89.8% recall. 41 of its 106 misses had one cause: Apertus labelled the party *recourant(e)*, and our check for public roles matched the substring *cour* (court) inside that word, so the party was kept. We fixed that one line (`role_redacts` in `src/anonymizer/llm.py`) and scored again with the same, cached Apertus answers. Because the bug was found on test, we also reported the dev split, which played no part in that fix: 90.8% recall, 0.0% of protected names redacted. Both test runs are in `docs/results/` (`eval_full_before_fix/`).

Two fixes made on dev only. After that run we read the 36 dev decisions and changed two things, without looking at test again: (1) Apertus returned doctors with their title attached ("Dr. med. Francesca Grassi", "docteur Julien Berset"), and our checks then dropped the name. Medical titles are now stripped like other titles, and a doctor the model would keep goes to *you decide* instead, because courts usually remove a private doctor's name but may keep an insurer's medical adviser. (2) Chambers, directorates and secretariats (*Kammer*, *Chambre*, *Camera*, *-direktion*, *Sekretariat* ...) were added to the public-body list, and a lone role word

(*Vermieterin*) is no longer read as a name. On dev, recall went from 90.8% to 93.9% and precision from 94.8% to 98.0%. We then scored test once: recall 94.7% → 96.1%, precision 92.4% → 95.5%, judge and lawyer names wrongly removed unchanged at 1.4%. No test decision got a new miss or a new false positive. The run before these fixes is in `docs/results/eval_full_before_dev_fixes/`.

The remaining 41 misses on test:

- *Lawyers the court had anonymised (17).* Now and then the court removes a lawyer's name too, when it would point to the client (`à Me A._____`). Our refill turns these into gold strings, while Redacta keeps every lawyer by design. The clerk has to catch these; a rule cannot.
- *Names the refill put in odd places (11).* The placeholder sometimes stood for a hospital, a company or a place ("Hôpital Camille Tissot", "in Urs Moser"), and Apertus did not list a person's name there. Part of this group is noise in our benchmark rather than a real leak.
- *Place names (10).* We inserted a town or street where the court had removed one ("un immeuble sis à Sion"). Neither Apertus nor the rules look for these.
- *Other (3):* a doctor written "la Dre" and labelled as an authority, a party read as a public service, a company read only in part.

47 false positives (81 before the dev fixes). They publish nothing wrong but cost the clerk a click each: public bodies still missing from our list (26, e.g. *Verwaltungsrekurskommission*, *État de Vaud*, *Unisanté*), common words read as names (7, e.g. *Gartenbaufirma*), judges or lawyers named later without a title (9, e.g. *Aubry Girardin*), and 5 addresses, dates or places.

6. Value and cost

Measured on the laptop used for this report (Apple M1 Max, 32 GB) with the weights `make model` downloads and the code at commit `f6bcf3b`, without cached answers (every Apertus call live):

Where Apertus runs	Decisions	Seconds per decision
llama.cpp with Metal (laptop GPU), all test decisions	144	**25.8**
llama.cpp with Metal, first 10 test decisions	10	27.2
`make run` container, CPU only (a Colima VM with 8 vCPUs and 10 GB), first 10 test decisions	10	110.4

The first 10 test decisions average 8,526 characters, longer than the test average of 6,459. On those ten, the CPU-only container is 4.1 times slower than Metal; with our own GGUF the same ten took 26.6 s and 109.5 s. Running continuously at 25.8 s per decision, one such machine with one model slot would process about 3,300 decisions a day; the CPU-only container about 780 decisions of that length. In 2025 the Federal Supreme Court delivered 7,883 judgments (joint press release of the federal courts, 17 March 2026). A first pass over a year of that court's output would take about 56 hours on one laptop GPU. The court's clerks would still review every decision; this figure is only the machine's part.

Costs: the software and the model weights are open; there is no per-call fee and no external API, and no text leaves the machine. The running cost is one machine with enough memory for a 5 GB model; on our laptop the GPU was about four times faster than the CPU-only container. We did not time clerks doing this work, with or without Redacta, so we make no claim about time saved per decision.

7. Limitations

- The benchmark comes from one court. Cantonal decisions have other layouts and more personal detail (health, family, places), which our rules do not cover. Before using it in a court, measure it on a few dozen of that court's own decisions, which `src/evaluate.py` can do.
- Synthetic names are common Swiss names, which is easier than rare or foreign names. Grammar sometimes no longer matches (a female name with a male pronoun), which may confuse the model slightly.

- We measure only what the court already removes (names and the identifiers we inserted). Indirect identifiers, such as a rare profession in a small village, are not in the answer key and are left to the clerk.
- A missed name is published if the clerk does not catch it. The tool is meant to be one step of the clerk's review, not to replace it.
- Speed: see *Value and cost*. The CPU-only container is slow enough that a court would want a GPU for daily use.

8. Reproducibility

- Hardware for the numbers: Apple M1 Max, 32 GB, macOS; llama.cpp `llama-server` (Metal), Q4_K_M, context 16,384 for the accuracy tables (8,192 for the community-weights run and the speed runs, as in the container), temperature 0.
- Our numbers come from `python -m src.evaluate` run directly on the Mac against `llama-server` with Metal (the same code as in the container). `make eval-all` runs the three setups in Docker and writes `outputs/eval_<mode>/metrics.json`, `predictions.jsonl` and `errors.jsonl`. With `LLM_CACHE_DIR` set, Apertus answers are cached, so a second mode reuses the first mode's calls.
- The benchmark is rebuilt deterministically with `make benchmark` (seed 7).
- Weights: the accuracy tables use the GGUF built by `scripts/convert_gguf.sh` from the official safetensors. `make model` downloads a public community conversion (`Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF`, revision `248ec68`, SHA-256 `a037df8d…f299`) with the same settings (Q4_K_M, end-of-generation id 68, same chat template; its vocabulary keeps 200 extra reserved rows). We downloaded it with `make model` into a fresh clone and ran the whole test split with it, with no cached answers: recall 96.2% (ours 96.1%), precision 94.1% (95.5%), leak-free 78.5% (77.8%), protected names wrongly removed 1.4% (1.4%). It misses 40 strings (ours 41) and adds 63 false positives (ours 47), mostly more courts, authorities and file numbers. Results: `docs/results/eval_full_community/`. The speed figures in *Value and cost* come from this run.
- Our outputs (metrics, every prediction, every error) are in `docs/results/`.

9. Next steps

- Measure on cantonal decisions with a partner court, and add their recurring patterns (land register numbers, plot numbers, vehicle data) to the rules.
- Use the clerk's corrections from `audit.jsonl` as training data for a small LoRA adapter on Apertus.
- Indirect identifiers: ask Apertus a second, separate question ("what in this text could identify a party to a neighbour?") and show the answers in *you decide*.
- Word/PDF import and export in the court's own template.

License

Code: Apache-2.0. Report and documentation: CC-BY-4.0. Data files: CDLA-Permissive-2.0.

References

- Swiss Federal Supreme Court, judgment 1C_394/2018 of 7 June 2019 (access to the decisions of the Geneva criminal court).
- J. Niklaus, R. Mamié, M. Stürmer, D. Brunner, M. Gygli. *Automatic Anonymization of Swiss Federal Supreme Court Rulings.* Natural Legal Language Processing Workshop (NLLP), 2023. arXiv:2310.04632.
- Swiss federal courts. Joint press release on the 2025 annual reports, 17 March 2026.
- spaCy `xx_ent_wiki_sm` multilingual NER model (evaluation baseline only), https://spacy.io/models/xx.
- A. Nyffenegger, M. Stürmer, J. Niklaus. *Anonymity at Risk? Assessing Re-Identification Capabilities of Large Language Models in Court Decisions.* Findings of NAACL, 2024.
- Swiss AI Initiative. *Apertus* model family, https://huggingface.co/swiss-ai.
- entscheidsuche.ch, open collection of Swiss court decisions.